Price Forecasting Model

End-to-End ML Pipeline Documentation
Training · Evaluation · Production Deployment

Model Type	SGDRegressor (Ridge-regularised Linear Model)
Target	avg_monthly_price
Train Metric	$R^2 = 0.9987$ | MAPE = 0.62%
Test Metric	$R^2 = 0.9913$ | MAPE = 0.68%
Safe Horizon	Next 1 month (lag-based, real prices only)
Files	model_pipeline.ipynb · training.ipynb · prediction.ipynb

Table of Contents

1. Project Overview
2. Dataset & Preprocessing
3. Feature Engineering
4. Models Evaluated
5. Final Model — SGDRegressor Pipeline
6. Training Results
7. Pipeline Architecture
8. Production Deployment
9. Model Limitations
10. Improvement Roadmap

1. Project Overview

This document describes the complete machine learning pipeline built to forecast average monthly commodity prices. The goal is to predict the price one month ahead using historical pricing patterns, seasonal signals, and momentum features. The project spans three Jupyter notebooks — model pipeline construction, model training and evaluation, and production prediction — each serving a distinct role in the lifecycle.

Objectives

- Accurately forecast next month avg_monthly_price using historical data.
- Build a reproducible, production-ready sklearn Pipeline with no data leakage.
- Save a fully self-contained model artifact for deployment with zero hardcoding.
- Document model limitations and define a clear improvement path.

2. Dataset & Preprocessing

The raw dataset (price_data.csv) contains monthly price observations indexed by date. Two preprocessing steps are applied before any feature engineering.

2.1 Outlier Handling — IQR Capping

Outliers are identified using the Interquartile Range (IQR) method and capped — not removed — to preserve time series continuity. The bounds are computed only on training data and stored inside the IQRClipper transformer so the same bounds are applied identically at prediction time.

```
Lower bound = Q1 - 1.5 x IQR | Upper bound = Q3 + 1.5 x IQR
```

Capping instead of dropping is critical for time series — dropping rows would break the continuity needed for lag and rolling features.

2.2 Why fit only on training data?

The IQRClipper is fitted exclusively on training data. This prevents data leakage — the model must not see any information from the test set during training. At prediction time, transform() is called (not fit_transform()), so the same training bounds apply to all new data.

3. Feature Engineering

Ten features are engineered from the raw price series. All features are computed inside the FeatureEngineer transformer, which is part of the preprocessing pipeline.

Feature	Type	Description
month	Calendar	Month number (1–12)
quarter	Calendar	Quarter number (1–4)
year	Calendar	Year number

trend	Time Index	Global row index — captures long-term direction
lag_1	Lag	Price from 1 month ago (most important feature)
lag_2	Lag	Price from 2 months ago
lag_3	Lag	Price from 3 months ago
roll_3	Rolling	3-month rolling mean of lagged price
month_sin	Cyclical	Sine encoding of month — captures seasonality
month_cos	Cyclical	Cosine encoding of month — captures seasonality
momentum	Momentum	Month-over-month price change (diff of lag_1)

NaN rows created by lag and rolling operations (first 3 rows) are dropped inside the transformer using dropna(). This is why the preprocessing pipeline cannot be merged with the model pipeline — dropping rows changes the shape, which breaks sklearn's row-alignment contract between X and y.

Why Sine/Cosine Encoding?

Month is a cyclical variable — December (12) is close to January (1), but a raw numeric encoding treats them as far apart. Sine/cosine encoding maps month onto a unit circle, so the model correctly understands that December and January are adjacent.

4. Models Evaluated

Several regression models were evaluated on the same train/test split (last 12 months as test). All models used the same feature set and StandardScaler normalisation.

Model	Train R^2	Test R^2	Test MAPE
SGDRegressor	0.9913	0.9913	0.68%
Prophet	0.8842*	0.1075	~24.5%
SARIMA	0.7610*	-1.4928	~32.1%

Why SGDRegressor was chosen

- Highest test R^2 (0.9913) with lowest test MAPE (0.68%) — best generalisation.
- L2 Ridge regularisation prevents overfitting without over-shrinking coefficients.
- Adaptive learning rate with early stopping avoids unnecessary iterations.
- Lightweight and fast — ideal for production deployment and monthly retraining.
- Tree-based models (RF, GBM) overfitted — high train R^2 but lower test R^2.

5. Final Model — SGDRegressor Pipeline

5.1 SGDRegressor Configuration

Hyperparameter	Value	Reason

loss	squared_error	Equivalent to Linear Regression MSE loss
penalty	l2	Ridge regularisation — penalises large weights
alpha	0.01	Moderate regularisation strength
max_iter	1000	Sufficient iterations for convergence
learning_rate	adaptive	Reduces LR when no improvement seen
eta0	0.1	Initial learning rate
early_stopping	True	Stops if validation score stops improving
validation_fraction	0.1	10% of train used for early stopping check
n_iter_no_change	20	Patience — 20 epochs of no improvement
random_state	42	Reproducibility
tol	1e-4	Convergence tolerance

5.2 Two-Pipeline Design

The architecture uses two separate sklearn Pipelines rather than one combined pipeline. This is a deliberate design choice driven by a fundamental sklearn constraint.

Pipeline	Steps	Purpose
preprocess Pipeline	IQRClipper → FeatureEngineer	Outlier capping + all feature creation + dropna
model Pipeline	StandardScaler → SGDRegressor	Normalisation + model training and prediction

The FeatureEngineer calls dropna() which reduces the number of rows. sklearn's Pipeline requires X and y to have the same row count at the model step — if dropna is inside the model pipeline, a shape mismatch error occurs. The two-pipeline design solves this cleanly: preprocess runs first, y is extracted from the cleaned output (already aligned), then the model pipeline trains on the aligned X and y.

---

6. Training Results

The model was evaluated on both the training set and a held-out test set consisting of the last 12 months of data — identical to real production conditions where the model forecasts one month at a time using real historical lags.

Metric	TRAIN	TEST	Interpretation
MAE	45.49	91.90	Average absolute price error
RMSE	87.86	124.67	Penalises large errors more heavily
MAPE	0.62%	0.68%	Less than 1% error — excellent accuracy
R^2	0.9987	0.9913	Model explains 99.1% of price variance

The small gap between train and test metrics indicates the model generalises well without significant overfitting. A MAPE below 1% is considered excellent for monthly commodity price forecasting.

Test Evaluation Method — Important Note

During test evaluation, all lag features (lag_1, lag_2, lag_3) are computed from real historical prices. Feature engineering runs on the full dataset before splitting, so the test rows already contain real observed lags. The model predicts all 12 test rows in a single batch call — no iterative prediction, no compounding errors. This is why test accuracy is high and matches production conditions for one-month-ahead forecasting.

7. Pipeline Architecture

Complete data flow from raw CSV to prediction:

Step	Component	Input	Output
1	Load CSV	price_data.csv	Raw DataFrame with DatetimeIndex
2	IQRClipper.fit	Train data only	Fitted lower/upper bounds stored
3	IQRClipper.transform	Full DataFrame	Outlier-capped price column
4	FeatureEngineer	Capped DataFrame	11 features + dropna applied
5	Train/Test split	Featured DataFrame	Last 12 rows = test set
6	StandardScaler	X_train features	Normalised features (mean=0, std=1)
7	SGDRegressor.fit	Scaled X_train, y_train	Trained model weights
8	Evaluate	X_train, X_test	MAE, RMSE, MAPE, R^2
9	joblib.dump	Both pipelines + metadata	price_model.joblib saved

What is saved in price_model.joblib

Key	Contents	Why Needed
preprocess	Fitted IQRClipper + FeatureEngineer	Transform new data identically to training
model	Fitted StandardScaler + SGDRegressor	Scale features and predict
last_3	Last 3 real price values [t-3, t-2, t-1]	Seed lags for next month prediction
last_trend	Final global trend index integer	Correct trend value for future rows
last_date	Date of last training observation	Compute next month date automatically

Feature names are NOT saved separately — they are extracted directly from model.named_steps['scaler'].feature_names_in_ which sklearn stores automatically when fitted on a pandas DataFrame. Zero hardcoding in prediction.

8. Production Deployment

8.1 Prediction Notebook (prediction.ipynb)

The prediction notebook has two parts. Part 1 is a model verification step that reloads the training data, applies the same split, and checks that metrics exactly match training — confirming the model loaded correctly. Part 2 predicts the next single month price.

8.2 Next Month Prediction Logic

A 4-row window is constructed from last_3 real prices plus the target future date. The FeatureEngineer computes all lag and rolling features from this window. The trend feature is manually overridden with last_trend + 1 to maintain the correct global position. The model then predicts a single price value.

```
seed_dates = last 3 months df_window = [price_t-3, price_t-2, price_t-1, price_t-1,
next_date] df_eng = engineer.transform(df_window) df_eng['trend'] = last_trend + 1
```

```
next_price = model.predict(df_eng[FEATURES].iloc[[-1]])
```

8.3 Monthly Retraining Workflow

As each new real price observation becomes available, the model should be retrained to maintain accuracy. This is the recommended production cycle:

Step 1: Month ends — new real price observed and added to price_data.csv.

Step 2: Re-run pipeline.py — IQRClipper refits on updated data, model retrains.

Step 3: New price_model.joblib is saved with updated last_3 and last_date.

Step 4: Run prediction.ipynb — Part 1 verifies accuracy, Part 2 predicts next month.

Step 5: The new real price replaces the previous prediction in lag history — no compounding error ever accumulates.

This monthly retrain cycle is the single most effective improvement. Each new real observation resets the lag anchor, keeping all predictions grounded in observed reality rather than compounded estimates.

---

9. Model Limitations

9.1 The Lag Compounding Problem

The model's three most important features are lag_1, lag_2, and lag_3 — the prices from the previous three months. During training and test evaluation, these lags are always real observed prices. But when predicting beyond one month into the future, the previous prediction must be used as the next lag, which itself feeds into the following prediction.

Forecast Month	lag_1 Source	lag_2 Source	Reliability
Month +1	Real price (last_3)	Real price (last_3)	High
Month +2	Predicted Month +1	Real price (last_3)	Medium
Month +3	Predicted Month +2	Predicted Month +1	Lower
Month +6	Predicted Month +5	Predicted Month +4	Low
Month +12	Predicted Month +11	Predicted Month +10	Very Low

WARNING: Multi-month forecasts from this model should not be used for business decisions. The model was designed and validated for one-month-ahead prediction only. Predictions beyond +1 month compound errors and produce increasingly unreliable results, as observed in tests showing straight-line drift over 60-month horizons.

9.2 Other Limitations

- Linear model — cannot capture sudden structural breaks or regime changes in price.
- No exogenous variables — supply shocks, geopolitical events, and demand shifts are invisible to themodel.
- IQR capping removes extreme price spikes — if a genuine spike occurs, the model will under-predict.
- Trend feature is a global index — assumes the long-term trend continues at the same rate.
- No confidence intervals — the model outputs a point estimate with no uncertainty range.

---

10. Improvement Roadmap

10.1 Short-Term Improvements (Immediate)

Improvement	Expected Benefit	Effort
Monthly retraining	Eliminates lag compounding entirely	Low
Add more lag features	Capture longer-term patterns (lag_6, lag_12)	Low
Add lag_12 (yearly)	Capture year-over-year seasonality	Low
Confidence intervals	Quantify prediction uncertainty	Medium

10.2 Medium-Term Improvements

Improvement	Expected Benefit	Effort
SARIMA / SARIMAX	Purpose-built for seasonal time series	Medium
Prophet (Meta)	Handles holidays, trend changes cleanly	Medium
Exogenous variables	Add supply/demand/macro economic features	High
Ensemble: Linear + SARIMA	Average predictions to reduce variance	High

10.3 Long-Term Improvements

Improvement	Expected Benefit	Effort
LSTM / Temporal Fusion Transformer	Learns complex non-linear temporal patterns	Very High
Probabilistic forecasting	Full distribution of future prices	High
Online learning	Model updates with every new observation	High
Automated retraining pipeline	MLOps — auto-retrain when drift detected	Very High

10.4 The Single Most Impactful Change

Monthly retraining with new real price data is the highest-ROI improvement. It requires no model changes — just run pipeline.py each month with the updated CSV. Each new real observation resets last_3, giving the next prediction perfect real lags instead of compounded estimates. This single change transforms the model from a one-shot static artifact into a continuously improving production system.

Price Forecasting ML Pipeline — End of Document

Files: model_pipeline.ipynb | training.ipynb | prediction.ipynb | price_model.joblib